

INTERFEJSY W JAVIE

Kontrakt · Wielodziedziczenie · Polimorfizm · Lambdy

Materiały do lekcji // Java SE 17+

7 rozdziałów

Teoria

Zastosowania

Java 8+

Skąd się wzięły interfejsy?

Żeby w pełni zrozumieć interfejsy, trzeba najpierw poczuć problem który rozwiązują. Nie są one dodatkowym ozdobnikiem języka — powstały jako odpowiedź na bardzo konkretne ograniczenie Javy.

Problem: Java nie ma wielodziedziczenia klas

W Javie klasa może dziedziczyć tylko po jednej klasie nadrzędnej. To celowe ograniczenie — wielodziedziczenie klas prowadzi do poważnych problemów nazywany problemem diamentu: gdy dwie klasy nadrzędne mają metodę o tej samej nazwie, kompilator nie wie którą wybrać.



Problem diamentu (diamond problem): klasa D dziedziczy po B i C, obie dziedziczą po A. Gdy A ma metodę metoda(), a B i C ją nadpiszą — która wersja trafi do D? Java unika tego problemu zabraniając wielodziedziczenia klas.

Rzeczywistość jest bardziej złożona

Ale rzeczywistość wymaga elastyczności. Kaczka jest ptakiem, ale też pływa. Dron nie jest ptakiem, ale też lata. Samochód amfibium jedzie i pływa. Żadna prosta hierarchia klas tego nie odwzoruje.

- Kaczka jest Ptakiem — dziedziczy po klasie Ptak
- Kaczka potrafi pływać — ale Samochód amfibium też potrafi
- Kaczka potrafi latać — ale Dron też potrafi, choć nie jest ptakiem



Wniosek: dziedziczenie opisuje czym coś jest. Interfejs opisuje co coś potrafi robić — niezależnie od hierarchii klas.

Interfejs jako rozwiązanie

Interfejsy pozwalają klasom deklarować zdolności niezależnie od dziedziczenia. Klasa może implementować dowolną liczbę interfejsów — nie ma tu żadnego ograniczenia. To eleganckie wyjście z pułapki pojedynczego dziedziczenia.

Czym jest interfejs?

Interfejs to kontrakt — formalna umowa między programistą a klasą. Gdy klasa implementuje interfejs, zobowiązuje się że dostarczy określone zachowania. Nie ma znaczenia jak je zrealizuje — ważne że je zapewnia.

Analogia z życia codziennego

Wyobraź sobie że zatrudniasz elektryka. Nie obchodzi cię czy skończył politechnikę czy kurs zawodowy, jakiej firmy narzędzi używa, ani jak dokładnie podłącza przewody. Obchodzi cię tylko że potrafi podłączyć instalację elektryczną. Interfejs `Elektryk` z metodą `podłączInstalacje()` opisuje właśnie tę zdolność.

Właściwości interfejsu

Nie można go instancjonować

Tak jak klasy abstrakcyjnej, nie da się napisać `new Latający()`. Interfejs istnieje wyłącznie jako kontrakt — nie jako obiekt.

Wszystkie metody są domyślnie abstrakcyjne

Przed Javą 8 interfejsy mogły zawierać tylko deklaracje metod bez ciała. Każda klasa implementująca musiała zdefiniować je samodzielnie. To zapewniało czysty kontrakt bez żadnej logiki.

Klasa może implementować wiele interfejsów

To kluczowa zaleta. Kaczka może jednocześnie implementować `Latający` i `Pływający`. Nie ma żadnych konfliktów, bo interfejsy nie mają implementacji — klasa sama decyduje jak każdą zdolność zrealizuje.

Interfejs może dziedziczyć po innych interfejsach

Interfejsy tworzą własne hierarchie przez słowo `extends`. Co ważne — jeden interfejs może rozszerzać wiele innych interfejsów. Tu wielodziedziczenie jest dozwolone, bo nie ma problemu z implementacją.

Cecha	Klasa abstrakcyjna	Interfejs
Instancjonowanie	NIE	NIE
Metody z implementacją	TAK (zwykle + abstract)	TAK (tylko default/static)
Pola	dowolne	tylko stałe (public static final)
Konstruktor	TAK	NIE
Dziedziczenie	extends — tylko jedna	implements — wiele naraz
Relacja	"jest" (tożsamość)	"potrafi" (zdolność)

Składnia — od zera do implementacji

Deklaracja interfejsu

Implementacja interfejsu

Wiele interfejsów jednocześnie

Interfejs + dziedziczenie razem

Klasa może jednocześnie dziedziczyć po jednej klasie i implementować wiele interfejsów. Kolejność: najpierw `extends`, potem `implements`.



Reguła kolejności: `extends` musi wystąpić przed `implements`. Można mieć tylko jedną klasę po `extends`, ale dowolnie wiele interfejsów po `implements`.

Interfejs jako typ — polimorfizm

Jedną z najpotężniejszych cech interfejsów jest możliwość użycia ich jako typu zmiennej. Dzięki temu kod nie musi wiedzieć z jakim konkretnym obiektem pracuje — tylko że spełnia on dany kontrakt.

Interfejs jako typ zmiennej

Interfejs jako parametr metody

To najważniejsze zastosowanie. Metoda przyjmująca interfejs jako parametr działa z każdą obecną i przyszłą klasą go implementującą — bez żadnych zmian w kodzie metody.

Polimorfizm przez interfejsy — przykład kompletny



Złota zasada: programuj do interfejsów, nie do konkretnych klas. Zamiast `ArrayList<String> lista = new ArrayList<>()` pisz `List<String> lista = new ArrayList<>()`. Dzięki temu możesz zmienić implementację nie ruszając reszty kodu.

Metody default i static (Java 8+)

Java 8 rozszerzyła interfejsy o dwa nowe rodzaje metod. Nie naruszają one czystości kontraktu, ale rozwiązują ważne problemy praktyczne.

Metoda default — implementacja w interfejsie

Metoda default ma pełną implementację w interfejsie. Klasa implementująca może ją nadpisać, ale nie musi — dostaje wtedy działającą wersję domyślną.



Po co metody default? Wyobraź sobie że masz interfejs z 50 implementacjami. Chcesz dodać nową metodę. Bez default — musisz dodać ją do wszystkich 50 klas. Z default — dodajesz raz w interfejsie, wszystkie klasy dostają działającą wersję.

Metoda static — narzędzia interfejsu

Metody static w interfejsie należą do interfejsu — nie do jego implementacji. Wywołujemy je przez nazwę interfejsu, nie przez obiekt. Służą jako metody pomocnicze powiązane tematycznie z interfejsem.

Konflikt metod default — co gdy dwa interfejsy mają tę samą metodę?

Gdy klasa implementuje dwa interfejsy z metodą default o tej samej nazwie, musi ją nadpisać — inaczej kompilator zgłosi błąd. Wewnątrz nadpisania można wywołać konkretną wersję przez `NazwaInterfejsu.super.metoda()`.

Interfejsy funkcyjne i wyrażenia lambda

Java 8 przyniosła rewolucję: jeśli interfejs ma dokładnie jedną metodę abstrakcyjną, można go zastąpić związłym wyrażeniem lambda zamiast anonimowej klasy. To interfejs funkcyjny.

Czym jest interfejs funkcyjny?

Interfejs funkcyjny (*functional interface*) to interfejs z jedną metodą abstrakcyjną. Może mieć dowolną liczbę metod default i static — to nie przeszkadza. Adnotacja `@FunctionalInterface` nie jest obowiązkowa, ale warto jej używać — kompilator pilnuje wtedy że nie ma więcej niż jednej metody abstrakcyjnej.

Lambda — skrócony zapis implementacji

Wbudowane interfejsy funkcyjne (`java.util.function`)

Java dostarcza gotowe interfejsy funkcyjne dla typowych operacji. Nie trzeba pisać własnych dla standardowych wzorców.

Interfejs	Metoda	Opis	Przykład lambda
Function	<code>R apply(T t)</code>	przekształca $T \rightarrow R$	<code>s -> s.length()</code>
Consumer	<code>void accept(T t)</code>	przyjmuje <code>T</code> , nic nie zwraca	<code>s -> System.out.println(s)</code>
Supplier	<code>T get()</code>	nie przyjmuje, zwraca <code>T</code>	<code>() -> "Hello"</code>
Predicate	<code>boolean test(T t)</code>	sprawdza warunek	<code>s -> s.isEmpty()</code>
BiFunction	<code>R apply(T t, U u)</code>	dwa argumenty $\rightarrow$ wynik	<code>(a,b) -> a + b</code>
Runnable	<code>void run()</code>	bez argumentów i wyniku	<code>() -> System.out.println("!")</code>

Zastosowania interfejsów w praktyce

Interfejsy to nie tylko teoria — są fundamentem praktycznie każdego poważnego projektu Java. Oto najważniejsze wzorce ich użycia.

1. Niezależność modułów

Dwa komponenty systemu mogą być pisane niezależnie, jeśli obaj programiści zgadzają się na interfejs między nimi. Jeden pisze 'co przyjmuje', drugi 'co dostarcza'. Zmiany po jednej stronie nie wpływają na drugą.

2. Łatwość testowania

W testach można podmienić prawdziwą implementację na uproszczoną (tzw. mock lub stub), bo obie implementują ten sam interfejs. Kod testowany nie widzi różnicy.

3. Wzorzec Strategy — wybór algorytmu w czasie działania

Interfejsy pozwalają zamieniać algorytmy bez zmiany kodu który je używa. To jeden z najważniejszych wzorców projektowych.

4. Interfejsy w kolekcjach Javy

Standardowa biblioteka Javy jest zbudowana na interfejsach. To dlatego możemy swobodnie zamieniać implementacje kolekcji.

Interfejs	Implementacje	Kiedy używać
List	ArrayList, LinkedList	Kolejność ma znaczenie, duplikaty OK
Set	HashSet, TreeSet, LinkedHashSet	Brak duplikatów
Map	HashMap, TreeMap, LinkedHashMap	Pary klucz-wartość
Queue	LinkedList, PriorityQueue	Kolejka FIFO / priorytety
Iterable	wszystkie kolekcje	Pętla for-each

5. Comparable i Comparator — porządkowanie obiektów

Dwa wbudowane interfejsy Javy służące do definiowania kolejności obiektów.

